P.Vishwa Nandan(192473015)

TASK 2 – FROM REQUIREMENT TO EVENT-DRIVEN DESIGN (8 Marks)

1. Aim

To design and implement a hospital outpatient appointment desk using Java Swing components, event listeners, and layout managers.

The application allows receptionists to enter patient details, select a department, choose a doctor and time slot, and book appointments with proper validation.

2. Problem Description

A hospital outpatient desk requires a graphical application for receptionists to book patient appointments.

The application must provide the following features:

1. Enter the patient ID.
2. Select a department from a list.
3. Display doctors based on the selected department.
4. Select an available appointment time slot.
5. Book the appointment.
6. Display a confirmation or error message.
7. Ask for confirmation before closing the application if a booking is in progress.

3. Task 2.1 – Model the Events (2 Marks)

3.1 Event Sources and Listeners

An event occurs when a user interacts with a GUI component, such as clicking a button or selecting an item from a list.

Java provides event objects and listener interfaces to handle these interactions.

Event Source	Event Object	Listener Interface	Action Performed
Patient ID TextField	ActionEvent	ActionListener	Accepts the entered patient ID
Department ComboBox	ActionEvent	ActionListener	Refreshes the doctor list
Doctor ComboBox	ActionEvent	ActionListener	Updates the selected doctor
Time Slot ComboBox	ActionEvent	ActionListener	Updates the selected time slot
Book Button	ActionEvent	ActionListener	Validates and books the appointment

Main Window	WindowEvent	WindowAdapter	Handles window closing

3.2 Explanation of Events

1. Patient ID Event

The receptionist enters a patient ID into a text field. When Enter is pressed, an ActionEvent is generated.

2. Department Selection Event

When the receptionist selects a department, an ActionEvent occurs. The application updates the doctor list based on the selected department.

3. Doctor Selection Event

The receptionist selects a doctor from the updated list.

4. Time Slot Selection Event

The receptionist selects an available appointment time slot.

5. Book Button Event

When the Book button is clicked, the application validates the patient ID, department, doctor, and time slot before booking.

6. Window Closing Event

When the receptionist clicks the window close button, a WindowEvent occurs. If a booking is in progress, the application asks for confirmation before closing.

4. Task 2.2 – Design the Component Hierarchy (2 Marks)

4.1 GUI Components Used

The application uses Java Swing components.

- JFrame – Main appointment window.
- JPanel – Organises the GUI components.
- JLabel – Displays field names and status messages.
- JTextField – Accepts the patient ID.
- JComboBox – Displays departments, doctors, and time slots.
- JButton – Starts the appointment booking process.
- JOptionPane – Displays confirmation, success, and error messages.

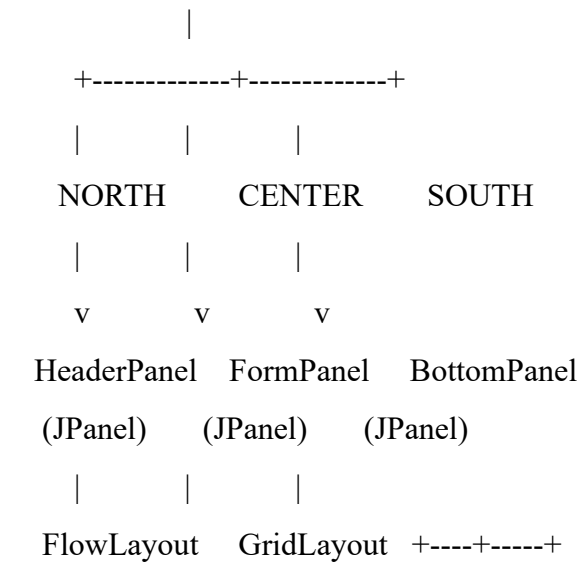
4.2 Component Hierarchy Diagram

AppointmentDeskFrame

(JFrame)

|

BorderLayout



4.3 Component Description

Component	Swing Class	Purpose
Main window	JFrame	Displays the application
Header	JPanel	Holds the hospital title
Form area	JPanel	Holds patient and appointment details
Patient ID	JTextField	Accepts patient ID
Department	JComboBox	Displays department names
Doctor	JComboBox	Displays doctors in the selected department
Time Slot	JComboBox	Displays appointment time slots
Book button	JButton	Books the appointment
Status message	JLabel	Displays application status
Confirmation	JOptionPane	Asks before closing or displays messages

4.4 Suggested Window Layout



Patient ID: [_____]

Department: [Select Department v]

Doctor: [Select Doctor v]

Time Slot: [Select Time Slot v]

[BOOK APPOINTMENT]

Status: Ready to book an appointment.

---

5. Task 2.3 – Design the Handler Logic (3 Marks)

5.1 (a) Pseudocode – Refreshing Doctor List When Department Changes

Aim: To display only the doctors belonging to the department selected by the receptionist.

Data Structure:

A Map<String, List<Doctor>> stores department names and their corresponding doctors.

Example:

Cardiology → [Dr. Ravi, Dr. Meena]

Neurology → [Dr. Kumar, Dr. Priya]

Orthopedics → [Dr. John, Dr. Latha]

Pseudocode:

START

WHEN department selection changes:

Get selected department

Clear existing doctor list

Search selected department in DoctorMap

IF department exists THEN

Get the doctors belonging to that department

FOR each doctor in the list

Add doctor to doctor ComboBox

END FOR

ELSE

Display "No doctors available"

END IF

Refresh doctor ComboBox

STOP

Explanation:

1. The receptionist selects a department.
2. The application retrieves the selected department.
3. The previous doctor list is cleared.
4. The application searches for doctors belonging to that department.
5. The matching doctors are added to the doctor ComboBox.

6. If no doctors are available, an appropriate message is displayed.

5.2 (b) Pseudocode – Validating and Booking the Appointment

Aim: To validate all required appointment details and book an appointment only when the information is valid.

Pseudocode:

START

WHEN Book button is clicked:

Get patient ID

Get selected department

Get selected doctor

Get selected time slot

IF patient ID is empty THEN

Display "Enter patient ID"

ELSE IF department is not selected THEN

Display "Select a department"

ELSE IF doctor is not selected THEN

Display "Select a doctor"

ELSE IF time slot is not selected THEN

Display "Select a time slot"

```
    ELSE
        Check whether appointment slot is available
        IF slot is available THEN
            Set booking status to in progress
            Save appointment details
            IF booking is successful THEN
                Display appointment confirmation
            ELSE
                Display booking error message
            END IF
            Set booking status to completed
        ELSE
            Display "Time slot is not available"
        END IF
    END IF
STOP
```

6. Java Implementation – Hospital Appointment Desk

The following program implements the appointment desk using Swing. It includes department selection, dynamic doctor lists, appointment validation, booking, and window-closing confirmation.

For demonstration, appointment data is stored in memory. A database can be integrated later.

6.1 Java Program – AppointmentDesk.java

6.2 Program Explanation

Step 1 – Creating the GUI

The JFrame creates the main appointment window. Panels organise the patient ID, department, doctor, and time-slot fields.

Step 2 – Loading Doctor Data

The loadDoctorData() method stores doctors in a Map based on their departments.

Step 3 – Selecting a Department

The department ComboBox generates an ActionEvent. The refreshDoctorList() method clears the old doctors and displays the doctors belonging to the selected department.

Step 4 – Validating the Input

The processBooking() method checks whether the patient ID, department, doctor, and time slot have been selected.

Step 5 – Booking the Appointment

The application uses SwingWorker to perform the booking in the background.

This prevents database or other slow operations from blocking the Event Dispatch Thread.

Step 6 – Preventing Duplicate Slot Booking

The AppointmentService uses a Set to check whether the selected doctor's time slot has already been booked.

If the slot is already occupied, the application displays an error message.

Step 7 – Closing the Application

The WindowAdapter checks whether a booking is in progress. If it is, the receptionist must confirm closing. The application closes after the ongoing operation finishes.

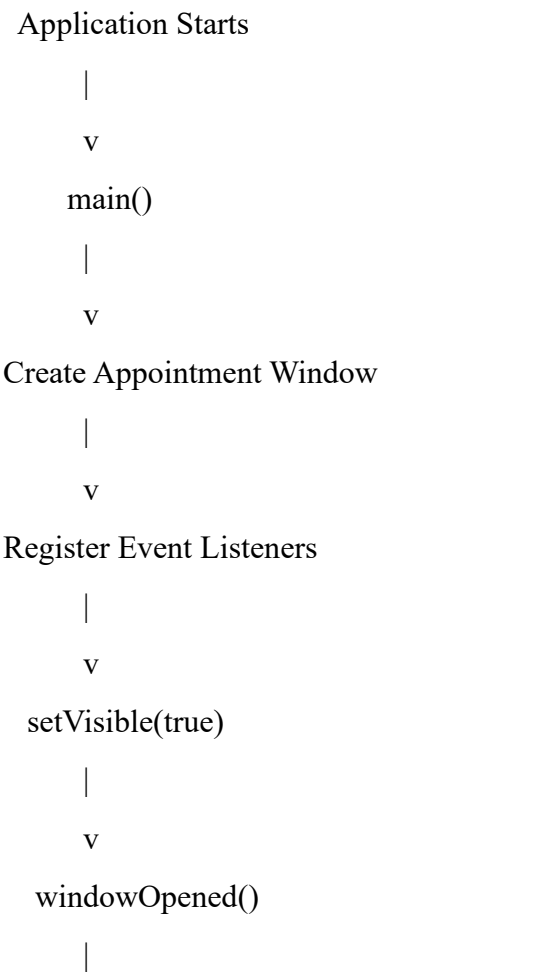
7. Task 2.4 – Trace the Event Sequence (1 Mark)

7.1 Event Sequence From Opening to Closing

1. The main() method starts the application.
2. SwingUtilities.invokeLater() schedules GUI creation.
3. The AppointmentDesk constructor creates the components.
4. Event listeners are registered.

5. setVisible(true) displays the window.
6. windowOpened() is triggered.
7. The receptionist selects a department.
8. An ActionEvent refreshes the doctor list.
9. The receptionist selects a doctor and time slot.
10. The receptionist clicks Book.
11. The ActionListener validates the information.
12. SwingWorker performs booking in the background.
13. The application displays a success or error message.
14. The receptionist clicks the close button.
15. windowClosing() checks whether booking is in progress.
16. The application asks for confirmation when necessary.
17. dispose() closes the window.
18. windowClosed() is triggered.

7.2 Event Sequence Diagram



v

Select Department

|

v

ActionEvent

|

v

Refresh Doctor List

|

v

Select Doctor and Slot

|

v

Click Book Button

|

v

ActionEvent

|

v

Validate Information

|

v

Start SwingWorker

|

v

Book Appointment

|

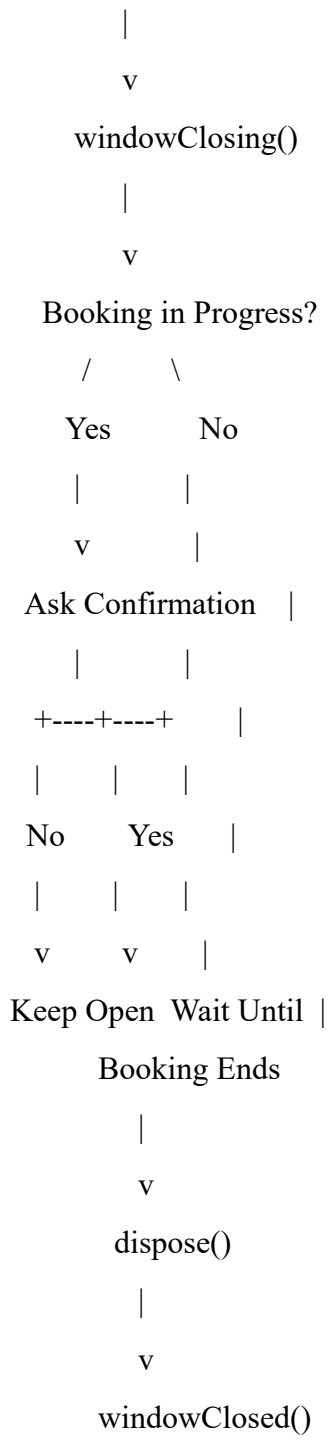
v

Show Success/Error Message

|

v

Click Window Close



7.3 Comparison With the Applet Life Cycle

Applet Method	**Purpose**
init()	Initialises the applet
start()	Starts or resumes the applet
paint()	Draws the applet contents
stop()	Stops or pauses the applet

destroy()	Releases applet resources

In a Swing desktop application:

- The constructor creates and initialises the GUI.
- setVisible(true) displays the window.
- windowOpened() handles the opening event.
- windowClosing() handles the closing request.
- dispose() releases the window resources.
- windowClosed() indicates that the window has been closed.

Swing applications use constructors and window events rather than the applet lifecycle methods.

8. Alternative Concept and Complexity Analysis

8.1 Alternative 1 – Fetch Doctors From the Database Every Time

Instead of storing doctor information in memory, the application can execute a database query whenever the receptionist selects a department.

SQL Query:

```
SELECT doctor_id, doctor_name

FROM doctors

WHERE department = ?;
```

Working:

1. The receptionist selects a department.
2. The application sends a query to MySQL.
3. The database retrieves matching doctors.
4. The application updates the doctor ComboBox.

Advantages:

- Displays recently updated doctor information.
- Does not need to store all doctors in memory.
- Suitable when doctor information changes frequently.

Disadvantages:

- Requires a database request every time the department changes.
- Network and database delays can slow the interface.
- Repeated queries increase database workload.

8.2 Alternative 2 – Loading Doctors Once Using Map

A Map<String, List<Doctor>> stores all department-wise doctor information when the application starts.

Example:

```
Map<String, List<Doctor>> doctorMap = new HashMap<>();


doctorMap.put(
   "Cardiology",
   Arrays.asList(
      new Doctor("D101", "Dr. Ravi"),
      new Doctor("D102", "Dr. Meena")
   )
);
```

When a department is selected, the application retrieves its doctors directly from the Map.

Example:

```
List<Doctor> doctors = doctorMap.get("Cardiology");
```

Advantages:

- Faster department selection after initial loading.
- Avoids repeated database queries.
- Reduces database workload.

Disadvantages:

- Requires additional memory.
- Doctor information may become outdated if database records change.
- Requires a refresh mechanism to update cached information.

8.3 Time Complexity Analysis

Let:

- d = Number of departments.
- m = Total number of doctors.
- m_i = Number of doctors in the selected department.
- Q = Database query and network processing cost.

Database Query Approach:

Approximate time per department change:

$O(Q + m_i)$

The database and network request introduce additional processing time. Without a suitable database index, the query may also need to scan many doctor records.

Map Approach:

Initial loading:

$O(d + m)$

Expected lookup time:

$O(1)$

Displaying the selected department's doctors:

$O(m_i)$

Approximate time per department change:

$O(1 + m_i)$

8.4 Space Complexity Analysis

Database Query Approach:

The application retrieves only the doctors belonging to the selected department.

Temporary memory:

$O(m_i)$

Map Approach:

The application stores all departments and their doctors in memory.

Space complexity:

$O(d + m)$

8.5 Comparison Table

Feature	**Database Query Every Time**	**Map Approach**
Initial loading	Minimal	Loads all departments and doctors
Department change	Database query	Map lookup
Time per change	$O(Q + m_i)$	$O(1 + m_i)$ expected
Additional application memory	$O(m_i)$	$O(d + m)$
Database workload	Higher	Lower after initial loading
Data freshness	More current	May become outdated

Response time	Depends on database	Usually faster

8.6 Engineering Decision

For the hospital appointment application, the Map approach is suitable when doctor information does not change frequently.

It allows quick doctor-list updates without repeatedly querying the database.

However, the application should refresh the Map when doctors are added, removed, or assigned to different departments.

9. Testing and Verification

Test Case	Input or Action	Expected Result
TC1	Enter valid patient ID	Patient ID accepted
TC2	Leave patient ID empty	Validation error displayed
TC3	Select Cardiology	Cardiology doctors displayed
TC4	Select Neurology	Neurology doctors displayed
TC5	Book without selecting a doctor	Error displayed
TC6	Book without selecting a time slot	Error displayed
TC7	Book with valid details	Appointment confirmation displayed
TC8	Book the same doctor's slot again	Duplicate booking rejected
TC9	Close while booking is running	Confirmation dialog displayed
TC10	Cancel closing	Window remains open

Expected Test Observation:

When Cardiology is selected, the doctor list should display Dr. Ravi and Dr. Meena.

When Neurology is selected, the doctor list should display Dr. Kumar and Dr. Priya.

A valid appointment should display a confirmation dialog. An already-booked time slot should be rejected.

10. Final Result

The hospital outpatient appointment desk was designed and implemented using Java Swing components and event-driven programming.

An event table was prepared to identify event sources, event objects, listener interfaces, and their actions.

The application uses JFrame, JPanel, JTextField, JComboBox, JButton, JLabel, and JOptionPane.

Department selection dynamically refreshes the doctor list. Input validation, appointment booking, duplicate-slot checking, background processing, and window-closing confirmation were implemented.

Finally, the database-fetching approach was compared with the Map-based approach using time and space complexity.

Thus, the application demonstrates event handling and GUI design for a hospital appointment system.